

Loops in Python

What are loops in programming?

Loops are programming structures that let us **repeat a block of instructions** automatically. Instead of writing the same code again and again, we write the rule once and let the loop execute it as many times as needed.

What is their role?

Their main role is **repetition and iteration**: processing many values, checking conditions repeatedly, traversing collections, validating input, automating tasks, and building algorithms that work across large amounts of data.

THE BASIC IDEA

START

begin task
choose data

REPEAT

run code
process value

CHECK

condition / next
value?

STOP

no more work
continue program

01 — LOOP FOUNDATIONS

Before learning syntax, understand the idea: a loop has a **repetition mechanism**, some **work** to perform, and a rule that tells Python when to move on or stop.

Without a loop, repetition looks like this:

```
python

print("Hello")
print("Hello")
print("Hello")
print("Hello")
```

OUTPUT

```
Hello
Hello
Hello
Hello
```

With a loop, the repeated rule is written once:

```
python

for _ in range(4):
    print("Hello")
```

OUTPUT

```
Hello
Hello
Hello
Hello
```

The important mental model is:

LOOP CYCLE

1. GET

next value
or state

2. RUN

body of
the loop

3. MOVE

update / next
iteration

4. CHECK

continue?
or stop?

A loop does **not** magically understand your intention. Your code must define what changes and what eventually makes the loop stop.

02 — FOR LOOP

Use a **for loop** when you want to go through items one by one, or when the number of repetitions is naturally expressed as a sequence.

```
python

for number in range(5):
    print(number)
```

OUTPUT

```
0
1
2
3
4
```

Read it in plain English: “For each number produced by `range(5)`, put that number into the variable `number` and execute the indented block.”

The variable name is your choice. `i` is common for a counter, but meaningful names are better when the value represents something.

● ● ● python

```
names = ["Ali", "Sara", "Hammad"]

for name in names:
    print("Hello", name)
```

OUTPUT

```
Hello Ali
Hello Sara
Hello Hammad
```

Here, the list itself supplies the values. The loop does not need a numeric counter.

FOR LOOP

COLLECTION

items
or range

CURRENT

one item
at a time

BODY

execute
your code

NEXT

move to
next item

03 — RANGE()

`range()` creates a sequence of integers commonly used with for loops. Its **stop value is excluded**.

● ● ● python

```
range(5)
# 0 1 2 3 4

range(2, 7)
# 2 3 4 5 6

range(2, 10, 2)
# 2 4 6 8

range(10, 0, -1)
# 10 9 8 7 6 5 4 3 2 1
```

Structure: `range(start, stop, step)`

- **start** = where to begin. If omitted, Python uses 0.
- **stop** = boundary that is not included.
- **step** = how much to move each time. If omitted, Python uses 1.

The most common beginner mistake is expecting `range(1, 6)` to include 6. It produces 1, 2, 3, 4, 5.

● ● ● python

```
for day in range(1, 6):  
    print("Day", day)
```

OUTPUT

```
Day 1  
Day 2  
Day 3  
Day 4  
Day 5
```

04 — FOR WITH STRINGS, LISTS & DATA

A for loop works with **iterables**: objects Python can move through one item at a time.

● ● ● python

```
word = "PYTHON"  
  
for letter in word:  
    print(letter)
```

OUTPUT

```
P  
Y  
T  
H  
O  
N
```

● ● ● python

```
scores = [72, 81, 65, 90]  
  
for score in scores:  
    if score >= 70:  
        print(score)
```

OUTPUT

```
72  
81  
90
```

This pattern is extremely important in Data Science: take a value → inspect it → apply a rule → keep/change/ignore it.

● ● ● python

```
expenses = [500, 1200, 300, 2500]  
  
for expense in expenses:  
    if expense > 1000:  
        print("Large:", expense)
```

OUTPUT

```
Large: 1200  
Large: 2500
```

05 — WHILE LOOP

Use a **while loop** when a condition should control how long the repetition continues. The number of iterations may not be known beforehand.

● ● ● python

```
number = 1

while number <= 5:
    print(number)
    number += 1
```

OUTPUT

```
1
2
3
4
5
```

Notice the three important pieces: initial state → condition → update.

WHILE CYCLE

STATE

number = 1

CHECK

number <= 5?

RUN

print number

UPDATE

number += 1

If you forget the update, the condition may remain True forever. That creates an **infinite loop**.

● ● ● python

```
password = ""

while password != "1234":
    password = input("Password: ")

print("Access granted")
```

06 — FOR vs WHILE

Both can repeat code, but they communicate different ideas.

Situation	Good fit	Why
Process every item in a list	for	The collection controls the iteration.
Repeat 10 times	for	range(10) expresses the count.
Ask until input is valid	while	The number of attempts is unknown.
Keep running while balance > 0	while	A condition controls duration.

Rule of thumb: If you can naturally say “for each item”, think **for**. If you naturally say “keep going while...”, think **while**.

07 — BREAK, CONTINUE & PASS

break stops the current loop immediately.

● ● ● python

```
for number in range(1, 11):  
    if number == 6:  
        break  
    print(number)
```

OUTPUT

```
1  
2  
3  
4  
5
```

continue skips the current iteration and moves to the next one.

● ● ● python

```
for number in range(1, 6):  
    if number == 3:  
        continue  
    print(number)
```

OUTPUT

```
1  
2  
4  
5
```

pass does nothing. It is useful when you need a syntactically valid block but have not written the logic yet.

● ● ● python

```
for i in range(5):  
    pass
```

08 — LOOPS + CONDITIONS

Most useful loops become more powerful when combined with **if / elif / else**.

● ● ● python

```
for number in range(1, 11):  
    if number % 2 == 0:  
        print(number)
```

OUTPUT

```
2  
4  
6  
8  
10
```

The loop visits every number. The condition decides which numbers are processed.

● ● ● python

```
scores = [45, 72, 91, 58, 80]

for score in scores:
    if score >= 70:
        print("Pass:", score)
    else:
        print("Fail:", score)
```

OUTPUT

```
Fail: 45
Pass: 72
Pass: 91
Fail: 58
Pass: 80
```

FILTER / DECIDE

LOOP

get next
value

CHECK

condition
is True?

YES

process
value

NO

skip / other
branch

09 — NESTED LOOPS

A **nested loop** is simply a loop inside another loop. It is not a new loop type.

The key rule: for every one outer iteration, the inner loop completes its entire cycle.

● ● ● python

```
for i in range(3):
    for j in range(3):
        print(i, j)
```

OUTPUT

```
0 0
0 1
0 2
1 0
1 1
1 2
2 0
2 1
2 2
```

Think row-by-row: outer loop chooses the row; inner loop moves through the values inside that row.

● ● ● python

```
for row in range(1, 4):
    for column in range(1, 4):
        print(row, column)
```

OUTPUT

```
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

If the outer loop runs **3** times and the inner loop runs **3** times each time, the inner body executes $3 \times 3 = 9$ times.

10 — NESTED LOOPS: PATTERNS

Nested loops are easiest to understand when you connect them to a visual structure.

Outer loop = rows • Inner loop = what appears inside each row

● ● ● python

```
for i in range(1, 6):
    for j in range(i):
        print("*", end=" ")
    print()
```

OUTPUT

```
*
**
***
****
*****
```

The inner loop runs 1 time on row 1, 2 times on row 2, 3 times on row 3, and so on.

● ● ● python

```
for i in range(1, 6):
    for j in range(1, 6):
        print(i * j, end="\t")
    print()
```

OUTPUT

```
1  2  3  4  5
2  4  6  8 10
3  6  9 12 15
4  8 12 16 20
5 10 15 20 25
```

This same row/column thinking appears later with matrices, tables, grids, image pixels, and many algorithms.

11 — LOOPS + NUMBER LOGIC

Loops give you a way to systematically test numbers. Combine them with operators such as **%**, **//**, and comparisons.

Prime-number check

● ● ● python

```
number = 29
is_prime = True

for i in range(2, number):
    if number % i == 0:
        is_prime = False
        break

if is_prime:
    print("Prime")
else:
    print("Not prime")
```

OUTPUT

Prime

Perfect-number check

● ● ● python

```
number = 6
total = 0

for i in range(1, number):
    if number % i == 0:
        total += i

if total == number:
    print("Perfect number")
else:
    print("Not perfect")
```

OUTPUT

Perfect number

Digit sum with while

● ● ● python

```
number = 12345
total = 0

while number > 0:
    digit = number % 10
    total += digit
    number //= 10

print(total)
```

OUTPUT

15

12 — REAL-WORLD PROGRAMS

Loops appear in programs where the user, data, or system can produce repeated work.

Input validation

● ● ● python

```
age = int(input("Enter age: "))

while age < 0:
    print("Invalid age.")
    age = int(input("Enter age: "))

print("Accepted")
```

Menu systems

● ● ● python

```
while True:
    print("1. Add")
    print("2. View")
    print("3. Exit")

    choice = input("Choose: ")

    if choice == "3":
        break

    print("Processing...")
```

Quiz / repeated questions

● ● ● python

```
score = 0

for question in range(3):
    answer = input("Answer: ")
    if answer == "python":
        score += 1

print("Score:", score)
```

13 — DATA SCIENCE & MACHINE LEARNING

Loops are a foundation for understanding data processing. Libraries such as pandas and NumPy later let you perform many operations more efficiently, often without writing explicit Python loops.

Filtering data

● ● ● python

```
scores = [72, 81, 65, 90]
passed = []

for score in scores:
    if score >= 70:
        passed.append(score)

print(passed)
```

OUTPUT

```
[72, 81, 90]
```

Logic: visit each record → test a rule → keep matching records.

Feature calculation

● ● ● python

```
heights = [1.70, 1.80, 1.65]
weights = [65, 80, 55]
bmi = []

for h, w in zip(heights, weights):
    bmi.append(w / (h ** 2))

print(bmi)
```

OUTPUT

```
[22.49..., 24.69..., 20.20...]
```

In ML, raw data is often transformed into features. The loop represents the basic idea of applying the same transformation to many records.

Matrix / tabular data

● ● ● python

```
dataset = [
    [10, 20, 30],
    [40, 50, 60]
]

for row in dataset:
    for value in row:
        print(value)
```

OUTPUT

```
10
20
30
40
50
60
```

Here the outer loop moves across rows and the inner loop moves across values in each row. This mental model becomes useful when you study matrices, tabular data, and algorithms.

Later concept: vectorization can perform many numerical operations without explicit Python loops. Learning loops first still matters because it teaches the underlying iteration logic.

14 — PRACTICE LADDER

Do these without looking at the solution first. Your goal is to predict the loop's state and output before running the program.

LEVEL 1 — BASIC FOR

Print 1–10. Print 10–1. Print even numbers from 1–20. Calculate the sum from 1–10.

LEVEL 2 — FOR + IF

Print multiples of 3. Count even numbers. Find numbers divisible by 5. Check whether a number is prime.

LEVEL 3 — WHILE

Print 1–10 with while. Make a countdown. Keep asking for a positive number. Build a password validator.

LEVEL 4 — CONTROL

Use break to stop at 50. Use continue to skip 5. Find the first number divisible by 7.

LEVEL 5 — NESTED

Make a star triangle. Make a square pattern. Build a 5×5 multiplication table. Trace every inner-loop execution.

LEVEL 6 — NUMBER LOGIC

Factorial. Digit sum. Reverse a number. Palindrome check. Perfect-number check.

LEVEL 7 — MINI DATA TASK

Given a list of expenses, calculate total, largest expense, count of expenses above a threshold, and filtered large expenses.

15 — QUIZ: PREDICT BEFORE YOU RUN

Write the output on paper first. Then run the code and compare.

Q1 What is printed?

● ● ● python

```
for i in range(3):  
    print(i)
```

Q2 How many times does the body execute?

● ● ● python

```
x = 1  
while x < 4:  
    print(x)  
    x += 1
```

Q3 Which value is skipped?

● ● ● python

```
for i in range(1, 6):  
    if i == 3:  
        continue  
    print(i)
```

Q4 How many total print executions occur?

● ● ● python

```
for i in range(2):  
    for j in range(3):  
        print(i, j)
```

Challenge: explain why the answer is correct, not just what the output is.

16 — LOOP CHEAT SHEET

```
# FOR – iterate through items
for item in items:
    process(item)

# FOR + RANGE – known repetitions
for i in range(5):
    print(i)

# WHILE – condition controls duration
while condition:
    do_something()
    update_state()

# BREAK – stop the current loop
for x in items:
    if stop_condition:
        break

# CONTINUE – skip this iteration
for x in items:
    if skip_condition:
        continue
    process(x)

# NESTED – loop inside another loop
for row in data:
    for value in row:
        process(value)

# COMMON DATA PATTERN
for value in values:
    if condition(value):
        process(value)
```

FOR → items / known repetitions

WHILE → condition-controlled repetition

BREAK → stop

CONTINUE → skip current iteration

NESTED → one loop inside another

The real skill is not memorizing loop syntax. It is being able to look at a problem and decide: **What repeats?**
What changes? What controls the repetition? When must it stop?

Next learning path: **functions** → **lists/dictionaries** → **exceptions** → **files** → **OOP** → **data structures** → **algorithms / DSA**.